Day 10 (Morning Session)

TypeScript Setup and Type Safety Fundamentals

Complete lecture notes · Instructor: Abhishek Purohit · Date: 30 October 2026

Prepared for: Sumeet Sharma

Topics covered in this lecture (in teaching order):

- Part 1 – Express: route prefixes with *app.use()* (product / admin routers) and shared middleware
- Part 2 – Why TypeScript? The type-safety problem of JavaScript; TypeScript is for development only; the TSC compiler
- Part 3 – Setting up TypeScript in a Node application (typescript, tsx, @types/node, dev dependencies)
- Part 4 – The tsconfig.json file (generating it, rootDir, outDir, strictness options, include / exclude)
- Part 5 – Type annotations for variables, functions, return types, void and arrays
- Part 6 – Object shapes with interfaces (optional properties, readonly, excess / missing property errors)
- Part 7 – Type aliases (the type keyword), literal-value types, interface vs type, naming conventions
- Part 8 – Union types
- Part 9 – Combining types to reuse properties (employee / student example)
- Part 10 – Revision material: summary, definitions, differences, mistakes, interview Q&A, exercises, checklist

ADDED EXPLANATION (not stated in the lecture)
How to read these notes. Normal text follows the lecture. Boxes labelled **ADDED EXPLANATION** are extra clarifications I added to help a beginner (they were not said by the instructor). Purple boxes are the instructor's key explanations, cleaned into readable English. Code that the instructor typed live is reproduced as shown in the lecture; where the transcript only described code (for example the Express route files) I reconstructed a small example and say so.

ADDED EXPLANATION (not stated in the lecture)
Transcript quality note. The transcript is an auto-generated mix of English and Hindi with many garbled phrases. Where a sentence was unclear, I used the surrounding explanation to establish the intended meaning (for example, the file is *tsconfig.json*, and the "combining types" section uses the intersection operator &). Exact numbers such as sample prices were not stated, so sample values in the code are illustrative.

Part 1 – Express: Route Prefixes and Shared Middleware

The session began by revisiting the Express application built the previous evening (29 October). That application already had authentication and a few APIs, protected by a *verifyToken* middleware. Before starting TypeScript, the instructor showed one improvement to the way routes are organised.

1.1 The starting point and its problem

Look at the product router. **Every single URL starts with the same prefix: /product.** If the product entity grows to fifty or a hundred URLs (and further operations), all of them will start with /product. Writing the prefix again and again on every route is repetitive and error-prone.

Before: prefix repeated on every route

```
// routes/productRouter.js   (reconstructed example - the lecture showed the same idea)
import express from "express";
import { verifyToken } from "../middleware/verifyToken.js";

const router = express.Router();

router.get("/product/list", verifyToken, (req, res) => {
    res.json([{ id: 101, name: "keyboard" }]);
});

router.post("/product/add", verifyToken, (req, res) => {
    res.json({ message: "Product added" });
});

router.get("/product/:id", verifyToken, (req, res) => {
    res.json({ id: req.params.id });
});

export default router;

// index.js
import express from "express";
import productRouter from "./routes/productRouter.js";

const app = express();
app.use(express.json());
app.use(productRouter);          // no prefix here

app.listen(3000);
```

1.2 The improved approach: mention the prefix once in app.use()

> **INSTRUCTOR'S WORDS (cleaned up)**
> If all the routes of an entity share the same prefix, we can mention that prefix **once** where the router is registered in index.js, right after the slash. Then the router file only has to contain what comes *after* /product. You can read the statement as: "*whenever a request arrives on the /product URL, this product router is going to do the job.*" For all requests arriving on /product, the product router responds.

Registering with a prefix

```
// index.js  (after)
app.use("/product", productRouter);      // prefix defined ONCE here
```

Routes inside the router now omit /product

```
// routes/productRouter.js  (after)
router.get("/list", verifyToken, (req, res) => {          // full URL: GET /product/list
    res.json([{ id: 101, name: "keyboard" }]);
});

router.post("/add", verifyToken, (req, res) => {          // full URL: POST /product/add
    res.json({ message: "Product added" });
});

router.get("/:id", verifyToken, (req, res) => {           // full URL: GET /product/:id
    res.json({ id: req.params.id });
});
```

How the request flows (execution order)

- A request **GET /product/list** arrives at the Express app.
- *app.use("/product", productRouter)* checks whether the URL starts with /product. It does, so the request is handed to productRouter.
- Express strips the matched prefix; inside the router the remaining path is **/list**.
- The route *router.get("/list", ...)* matches and the handler runs.

COMMON MISTAKE
After moving the prefix into *app.use()*, you must **remove** /product from every route inside the router. If you keep both, the URL becomes /product/product/list and the old URL stops working.

1.3 The same technique for the admin router

The admin router had the same situation: every URL started with /admin. The instructor applied the identical change - write **/admin** once in index.js, and inside the admin router only define what comes after /admin.

```
// index.js
app.use("/product", productRouter);
app.use("/admin", adminRouter);
```

INSTRUCTOR'S WORDS (cleaned up)
"This is one more thing we can do to customise all the endpoints." The prefix moves out of every route and into the single *app.use()* line.

1.4 Applying middleware once (verifyToken)

One more observation: **each and every route of the product URL was secured with the verifyToken middleware.** That job can also be done a single time, again inside *app.use()*, by mentioning the middleware there. Then the middleware no longer needs to be repeated on every route. The instructor described this as the flexibility available, and then said "as of now, let's keep it till the URL prefix only" and moved on.

```
// Idea mentioned in the lecture (not run in class):
app.use("/product", verifyToken, productRouter);   // protects ALL /product routes

// productRouter.js - routes no longer repeat verifyToken
router.get("/list", (req, res) => { res.json([]); });
```

ADDED EXPLANATION (not stated in the lecture)
Order matters. Express runs middleware in the order it is registered. In *app.use(path, verifyToken, router)*, verifyToken runs first, and only if it calls *next()* does the router run. Use this only when **every** route of that router needs the check; if some routes must stay public (like login), keep them in a different router.

Before vs after - quick comparison

Aspect	Before	After
Prefix (/product)	Typed on every route	Written once in app.use('/product', ...)
Route path in router	'/product/list'	'/list'
Changing the prefix later	Edit every route	Edit one line
Risk of typo	High (repeated many times)	Low

Part 2 – Why TypeScript?

The instructor then introduced "one more concept that is going to be there in every real-time application": **TypeScript**. A new folder for the 30 October morning session was created for the demos.

2.1 Where JavaScript is today

We already know JavaScript. We can run it in the browser, run it on the server with Node.js (so we can create a whole server in JavaScript), and later with React or Angular we can build the entire front end. So JavaScript is already very capable. **But its major drawback is type safety.**

2.2 The type-safety problem in JavaScript

INSTRUCTOR'S WORDS (cleaned up)
JavaScript is **not** a type-safe language. You can declare a variable, but you cannot assign it a data type. By mistake, the variable may get a different type of value - and JavaScript will not throw any error. It simply accepts the new value and the variable gets the new data type.

What JavaScript **cannot** do (as listed in the lecture):

- You cannot give a **data type to variables**.
- You cannot give **data types to function arguments**.
- You cannot give a **return type to functions**.
- When you create a JSON object you cannot decide its **shape** - which properties are required and which are optional.
- Certain **object-oriented features** were already missing from JavaScript.

Illustration of the problem (added example)

```
// JavaScript - everything below is accepted silently
let x = 5;
console.log(x);          // 5
x = "hello";             // no error! x was a number, now it is a string
console.log(x);          // hello

function sum(a, b) { return a + b; }
console.log(sum(3, 8));      // 11
console.log(sum("3", 8));    // "38"  <- string concatenation, not addition
```

Expected output

```
5
hello
11
38
```

ADDED EXPLANATION (not stated in the lecture)
Why does *sum("3", 8)* give 38? JavaScript converts 8 to a string and joins them. In a large application such a silent mistake can travel far before anyone notices. TypeScript catches it while you are writing code.

INSTRUCTOR'S WORDS (cleaned up)
It is **never recommended to write an enterprise application in plain JavaScript**, because of this one major drawback: type safety. We should write code in such a way that type safety imposes certain rules and regulations through data types.

2.3 What is TypeScript?

INSTRUCTOR'S WORDS (cleaned up)
TypeScript is nothing but **JavaScript with added support for data types**. It is not a totally new language. Everything that exists in JavaScript - console.log, let, var, const, functions, promises - is already there in TypeScript. The only new thing is that now we also deal with data types.

With TypeScript you can:

- Give a **data type to a variable** when you declare it.
- Specify the **type of each argument** and the **return type** of a function.
- Decide the **shape of a JSON object** (its properties).
- Use **interfaces** to describe what properties and methods a class should have.

2.4 TypeScript is for development only

INSTRUCTOR'S WORDS (cleaned up)
We need type safety **at development time**, on the developer's machine. We do not need it for execution in production. As a developer you write the entire Node application in TypeScript. But for deployment - whether the test environment, staging for a customer demo, or the live production environment - you do **not** deploy the TypeScript code. You **compile** it into JavaScript, create a build, and deploy that resulting JavaScript.

Technically you *can* deploy TypeScript code, but you **should not**. The reason given: if TS code is deployed, it has to be recompiled to JS dynamically at run time before it executes, so you would waste time on each and every request and execution. Compile once, then deploy only the JS.

The workflow

```
Write code            Compile                Deploy
----------   --->   ------------   --->   ----------------
.ts files           tsc (compiler)         .js files only
(developer          produces .js           (test / staging /
 machine)           in a build folder       production servers)
```

Question	Answer	Key point
Where do we write TypeScript?	On the developer machine	Gives type safety while developing
What do we deploy?	The compiled JavaScript	TS code is not deployed to test / staging / prod
Why not deploy .ts directly?	It would be recompiled at run time for each execution	Wastes time; compile once and deploy JS instead
What converts TS to JS?	The TypeScript compiler, **tsc**	Comes with the typescript package

2.5 The TypeScript compiler: tsc

TypeScript comes with its own compiler. The compiler reads your TypeScript code and produces the resulting JavaScript, which can be executed and deployed. **The name of the compiler is tsc.**

INSTRUCTOR'S WORDS (cleaned up)
Analogy given: when you install the Java JDK you also get the Java compiler, named **javac**. In the same way, installing TypeScript gives you the compiler named **tsc**. (JavaScript itself has no compiler, but TypeScript does.)

The instructor recommended building applications - back end or front end - in TypeScript.

Part 3 – Setting Up TypeScript in a Node Application

To develop with TypeScript we need an ecosystem: a **compiler**, a **type checker** and a couple of other things. So first we install a few dependencies.

3.1 The three dependencies and their purpose

Package	What it provides	Purpose / key point
typescript	The **tsc** compiler and the **type checker engine**	The core module for TypeScript. Required.
tsx	An engine that runs TS code directly on the developer machine, compiling it internally	Used only on the dev machine. **Optional** - saves manual compile-then-run steps.
@types/node	Type definitions for Node's built-in modules (http, fs, os, path ...)	Node's built-in modules only ship JS functions; TypeScript needs their type definitions.

Why do we need tsx?

To run TypeScript you would normally first compile with tsc and then run the resulting JS. During development we change code a thousand times a day, so *change code → compile → run* every time is tedious. **tsx** removes this: you just run your TS file and tsx compiles it behind the scenes. Changes are reflected automatically without manual compilation.

> **INSTRUCTOR'S WORDS (cleaned up)**
> Using tsx is **totally optional**. If you opt out of it you must do the compilation yourself. When an application is very small, generating a build manually is feasible. When the application scales, building every time takes a lot of time and is a tedious process - that is why we simplify development with tsx. tsx is for the development machine; it should not be used in production because it compiles behind the scenes and would increase run-time cost.

Why do we need @types/node?

Node has predefined built-in modules (fs, http, os, path, ...). Their functions exist only as JavaScript, without type definitions. Since TypeScript works with data types, we need type definitions for the functions inside these modules. The package **@types/node** installs them all.

3.2 Dependencies vs devDependencies

The instructor then asked the class: *are we going to keep these TypeScript dependencies on the live server?* The answer was **no**.

> **INSTRUCTOR'S WORDS (cleaned up)**
> TypeScript is meant only for development. When we deploy, the compiled JS runs; the TypeScript, tsx and @types/node packages are not needed. If we install everything as normal dependencies, they will also be installed on the server and occupy space. The **node_modules folder is very expensive** - thousands of files and folders - and on a server you pay for every byte of storage. So we mark development-only packages as **dev dependencies**.

> **INSTRUCTOR'S WORDS (cleaned up)**
> At deployment time, a DevOps engineer (or a Docker-style script) simply leaves out the dev dependencies and installs only the normal dependencies.

Kind	Meaning	Examples from the lecture
dependencies	Needed to run the application, including on production. Without them the code cannot execute.	express, mysql2, bcrypt, jsonwebtoken

Kind	Meaning	Examples from the lecture
devDependencies	Needed only on the developer machine. Left out of production installs.	typescript, tsx, @types/node, ESLint (checks code quality and whether good practices / rules were followed)

To mark a package as a dev dependency we add the flag **--save-dev** to the install command.

> **ADDED EXPLANATION (not stated in the lecture)**
> **--save-dev** has the short form **-D**. Also, on production servers dev dependencies are typically skipped with *npm install --omit=dev* (or *npm ci --omit=dev*). This is the added detail behind the "script that leaves dev dependencies behind" mentioned by the instructor.

3.3 Step-by-step project setup (done live)

Step 1 – Create the folder and open it in VS Code

A folder named **typescript-demo** was created (inside the 30 October morning folder) and opened in VS Code.

Step 2 – Create package.json

```
npm init -y
```

npm init creates the package.json file; **-y** accepts all default values.

Step 3 – Switch the project to ES modules

Inside package.json, the instructor removed *commonjs* and set the type to **module**:

package.json (type changed to module)

```
{
  "name": "typescript-demo",
  "version": "1.0.0",
  "type": "module"
}
```

Step 4 – Install the dev dependencies

```
npm i typescript tsx @types/node --save-dev
```

This creates node_modules and package-lock.json, and records the packages in package.json under the key **devDependencies**.

package.json after installation (version numbers depend on the day you install)

```
{
  "name": "typescript-demo",
  "version": "1.0.0",
  "type": "module",
  "devDependencies": {
    "@types/node": "...",
    "tsx": "...",
    "typescript": "..."
  }
}
```

Later, if you install runtime packages such as *npm i express mysql2*, they are recorded under **dependencies** (they are needed locally **and** on the live server). So package.json now has both *dependencies* and *devDependencies*.

KEY POINT
All the files still go into the same **node_modules** folder. The marking only matters for deployment: the DevOps engineer can leave out the dev dependencies and install just the dependencies on the production environment. Marking dependency type is a good practice we should follow.

Part 4 – The tsconfig.json File

You can start writing TypeScript right after installing the packages, but in a Node application that uses TypeScript you must have one more file: **tsconfig.json**.

4.1 What is tsconfig.json and why do we need it?

INSTRUCTOR'S WORDS (cleaned up)
The tsconfig file is used to **control the behaviour of the TypeScript compiler**. It decides how your TypeScript compiler is going to work: which files to include in compilation, which to exclude, where the output goes, and which rules and regulations to apply. Configurations are written as a JSON object.

Examples given:

- Your application will contain **.ts** files (index.ts, controller.ts, productRouter.ts, verifyToken.ts) instead of .js files. The config says which folders to **include** in compilation and which to **exclude** (exclude node_modules, include src).
- Where the compiled JavaScript should be placed in the project.
- How strictly the compiler checks your code.

4.2 Creating the file: use tsc --init, not manual creation

You could create *tsconfig.json* manually, but it is **not advised**. Instead, let the TypeScript compiler generate it.

Attempt 1 – running tsc directly (this fails)

```
tsc --init
```

Editor / compiler error

```
tsc : The term 'tsc' is not recognized as the name of a cmdlet, function, script file, or operable program.
```

Why does this error happen?

INSTRUCTOR'S WORDS (cleaned up)
TypeScript is installed **locally** inside our project, not globally. So the TSC compiler lives in the project's **node_modules** folder. But when you type a command in the terminal, the system looks for the tool at the **global** location. It does not find tsc there, hence the error. The mismatch is: installed locally, but searched globally. We must force the system to look for tsc inside the current project's node_modules folder.

Attempt 2 – use npx (this works)

```
npx tsc --init
```

Expected output

```
Created a new tsconfig.json
```

npx makes the system look for the command inside the current project's node_modules (the local installation) instead of only at the global location. The command *npx tsc --init* generates a new **tsconfig.json** file in the project's root, visible in the folder structure.

Command	Result	Reason
tsc --init	Error: 'tsc' is not recognized	Terminal searches the global location; TypeScript is installed locally
npx tsc --init	Creates tsconfig.json	npx forces the lookup inside the project's node_modules

ADDED EXPLANATION (not stated in the lecture)
Do we need to install tsc separately? **No.** The instructor confirmed that installing the *typescript* package installs tsc automatically; it is a command-line (CLI) tool. If you want to browse all options, the generated file links to the official documentation (tsconfig reference) - the instructor pointed to it for the full list of settings.

4.3 Inside tsconfig.json: compilerOptions

The generated file contains one JSON object. The key **compilerOptions** holds key-value settings; all keys are **predefined**. Some settings are active and many are commented out.

Typical tsconfig.json for this lecture (the exact generated file varies by TypeScript version)

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",

    "module": "nodenext",
    "target": "esnext",

    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitReturns": true,
    "noImplicitOverride": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "noPropertyAccessFromIndexSignature": true
  },
  "include": ["src"],
  "exclude": ["node_modules"]
}
```

ADDED EXPLANATION (not stated in the lecture)
tsconfig.json allows comments, which is why the generated file can contain commented-out options. The exact defaults produced by *tsc --init* depend on your TypeScript version, so your generated file may differ slightly from the one above. The settings below are the ones the instructor discussed.

(a) rootDir – where your source code lives

rootDir specifies the root folder of your TypeScript source code. We create a folder named **src** and keep the entire source code there: index file, routers, controllers, config, middlewares, services, and so on. So we set "*rootDir*": "*./src*".

(b) outDir – where the generated JavaScript goes

outDir decides where the resulting JavaScript is placed. Good practice is to name this folder **dist** (short for *distributable*). When compilation runs, a new *dist* folder is created at the root level, containing the generated JavaScript files. This is what you take forward for deployment.

How rootDir affects the output folder structure

INSTRUCTOR'S WORDS (cleaned up)
When TypeScript compiles, it keeps the **same directory structure** in the output directory, **relative to rootDir**. We specify rootDir so that the generated JavaScript has the same file-and-folder organisation as the TypeScript source - but without the extra top-level source folder.

Instructor's example, redrawn

```
my-project/
  tsconfig.json
  core/
    a.ts
    b.ts
    sub/
      c.ts

With  "rootDir": "./core", "outDir": "./dist"
  dist/
    a.js
    b.js
    sub/
      c.js

With  "rootDir": "./"
  dist/
    core/
      a.js ...
```

With *rootDir* pointing at the source folder, **dist** directly contains a.js, b.js and sub/c.js - the source folder itself is not recreated. For our project, with rootDir = ./src, the file *src/index.ts* will become *dist/index.js*.

(c) module and target

module and **target** tell the compiler which JavaScript standard to generate. The instruction: always go with the **latest ECMAScript standards**, so **leave them as they are** (predefined values).

(d) Strictness and quality options

The compiler can read your code with *stricter type checking*. Each of these settings can be turned on or off, which lets you decide how strict or lenient the compiler is.

Option	What it does	Lecture example
noUncheckedIndexedAccess	Warns when you access an array / index that may not exist	An array has a few elements and you try index 10 - you get a warning when the setting is on; with false, no warning
noUnusedLocals	Warns about unused local variables	Unused local variable gives a warning / error
noUnusedParameters	Warns about unused function parameters	Unused function parameter is flagged
noImplicitReturns	Every code path of a function must return a value (error otherwise)	Functions where some paths do not return raise an error
noFallthroughCasesInSwitch	Warns when a switch case falls through to the next one	A missing *break* in a switch case raises a warning
noImplicitOverride	Requires the *override* keyword when a subclass overrides a method	Listed in the generated file (not detailed in class)
noPropertyAccessFromIndex Signature	Forces bracket access for properties coming from index signatures	Listed in the generated file (not detailed in class)
exactOptionalPropertyTypes	Optional properties may be omitted, but cannot be explicitly set to *undefined*	Discussed in Part 6 (optional properties)

ADDED EXPLANATION (not stated in the lecture)
The explanations for noImplicitOverride, noPropertyAccessFromIndexSignature, and the precise wording of the other rows are added to help beginners; in class the instructor named these options and explained the idea of strictness and turning each on or off.

4.4 include and exclude

Apart from *compilerOptions*, the config can say which folders the compiler should consider. Since a real application compiles in a single go, we specify:

```
"include": ["src"],
"exclude": ["node_modules"]
```

- **include** – folders considered for compilation. We write **src**.
- **exclude** – folders left out. We must exclude **node_modules**; otherwise the compiler would try to compile it too, which we do not want.
- The instructor also mentioned the general recommended pattern **src/**/*** (double asterisk then single asterisk, meaning every file inside src and its sub-folders), but for the demo wrote only *src*.

INSTRUCTOR'S WORDS (cleaned up)
The compiler always **reads the tsconfig file first**; it then behaves according to those configurations. All settings related to the compiler's behaviour live in this one file.

4.5 The rule for source files

KEY POINT
All source code must live inside the **src** folder and must have the **.ts** extension - index file, controller, service, data layer, middleware, anything. The instructor answered the student question "can we use TypeScript across all files instead of JavaScript?" with: **we must**.

Project structure at this point

```
typescript-demo/
  node_modules/
  src/
    index.ts          <- all TypeScript source here
  package.json
  package-lock.json
  tsconfig.json
  dist/               <- created later by compilation (JavaScript output)
```

Part 5 – Type Annotations: Variables, Functions, Arrays

Now the TypeScript code itself. A file **src/index.ts** was created. A **type annotation** is the label you attach to a variable, parameter or function result to say what kind of value it holds. In TypeScript you write the annotation **after a colon**.

5.1 Typing variables

Purpose: show that a variable declared with a type will not accept a value of another type. **Problem solved:** the silent type change seen in JavaScript.

```
// src/index.ts
let x: number = 5;
console.log(x);

x = "hello";            // ERROR: string assigned to a number variable

let y: string = 6;      // ERROR: number assigned to a string variable
console.log(y);
```

Editor / compiler error

```
Type 'string' is not assignable to type 'number'.
Type 'number' is not assignable to type 'string'.
```

Line-by-line explanation

- **let x: number = 5;** - *let* declares the variable; **: number** is the type annotation; = *5* gives it a value. After this, x can only ever hold numbers.
- **console.log(x);** - prints the value of x.
- **x = "hello";** - tries to put a string into a number variable. The editor underlines it: *Type* '*string*' *is not assignable to type* '*number*'.
- **let y: string = 6;** - the type is string but the value is a number, so the error becomes *Type* '*number*' *is not assignable to type* '*string*'.

The valid types include *number*, *string*, *boolean* and others - "it can be any particular type". We now have **one level of type safety**.

Corrected version and output

```
let x: number = 5;
console.log(x);

let y: string = "six";
console.log(y);
```

Expected output

```
5
six
```

ADDED EXPLANATION (not stated in the lecture)
Important detail about tsx. tsx runs your code quickly but does **not** perform type checking - it simply strips the type annotations. So a file containing a type error may still run with tsx. The red underlines in VS Code come from the TypeScript type checker. To check types from the terminal, run *npx tsc --noEmit* (checks without producing output files). The instructor's next session covers running, compiling and building.

5.2 Typing functions: parameters and return type

Purpose: a function *sum* that takes two numbers and returns their sum, and a function *show* that prints a message and returns nothing. **Problem solved:** calling a function with the wrong kind of arguments, or returning the wrong kind of value.

```
function sum(a: number, b: number): number {
    return a + b;
}

function show(): void {
    console.log("Hello from TypeScript");
}

const rs: number = sum(3, 8);
console.log(rs);
show();

sum("3", 8);          // ERROR
```

Editor / compiler error

```
Argument of type 'string' is not assignable to parameter of type 'number'.
```

Expected output (after removing the erroneous line)

```
11
Hello from TypeScript
```

Explanation in logical blocks

- **a: number, b: number** - each *parameter* (argument) gets its own type after a colon.
- **): number** - after the closing parenthesis of the arguments, a colon introduces the **return type**. Here the function must return a number.
- **: void** - for a function that returns nothing, you can explicitly write the return type **void**.
- **const rs: number = sum(3, 8);** - you can also annotate the variable receiving the result. If the type were wrong the compiler would report an error.
- **sum("3", 8)** - passing a string where a number is expected is rejected: the function **always accepts numbers only**.

Execution flow

- sum(3, 8) is called; a = 3, b = 8; it returns 11; 11 is stored in rs.
- console.log(rs) prints 11.
- show() runs and prints the message. It returns nothing (void).

Syntax	Meaning	Example
variable: type	Type of a variable	`let x: number = 5;`
param: type	Type of a function argument	`function f(a: number)`
): type	Return type of a function	`function f(): number`
): void	Function returns nothing	`function show(): void`

5.3 Typing arrays

A normal array can hold any type of element. If you want an array to hold **only numbers**, give it an array type: the element type followed by **[]**.

```
// Without a type: mixed values are allowed
let mixed = [10, 20, 30, "hello"];

// With a type: only numbers allowed
let arr: number[] = [10, 20, 30];
arr.push(40);          // OK
arr.push("hello");     // ERROR
```

Editor / compiler error

```
Argument of type 'string' is not assignable to parameter of type 'number'.
```

ADDED EXPLANATION (not stated in the lecture)

number[] means "an array of numbers". An equivalent spelling is *Array<number>*. Similarly *string[]* is an array of strings. This notation is reused in Part 6 for arrays of objects.

Part 6 – Object Shapes with Interfaces

6.1 The problem: nothing controls the shape of an object

Suppose we hold data for a product. In plain JavaScript we just create a JSON object with key-value pairs, for example name = keyboard, price, quantity. Nothing stops us from forgetting a key or adding a wrong one. With TypeScript we can restrict the **shape** of the object - which keys it must have and their types.

> **INSTRUCTOR'S WORDS (cleaned up)**
> Along with primitive types (number, string, boolean) we can define our **own** data types. To give a shape to an object, we use the concept of an **interface**.

6.2 Defining an interface

```
interface Product {
    id: number;
    name: string;
    price: number;
    quantity: number;
}

const product: Product = {
    id: 101,
    name: "keyboard",
    price: 1500,
    quantity: 10
};
console.log(product);
```

Expected output

```
{ id: 101, name: 'keyboard', price: 1500, quantity: 10 }
```

In the whole application there is now **consistency**: every product object *must* have id, name, price and quantity with those types. (Sample values such as 1500 are illustrative.)

What if a required property is missing?

```
const product: Product = {
    id: 101,
    name: "keyboard",
    price: 1500
    // quantity is missing
};
```

Editor / compiler error

```
Property 'quantity' is missing in type '{ id: number; name: string; price: number; }' but required in type 'Prod
```

> **INSTRUCTOR'S WORDS (cleaned up)**
> This is how TypeScript **prevents bugs early**. In JavaScript we might end up forgetting the quantity key without any notice. TypeScript raises an error, and we are forced to create that property in the object.

6.3 Arrays of objects

To hold several products, create an array and give it the type **Product[]** - an array that contains only Product objects.

```
const productsList: Product[] = [
    { id: 101, name: "keyboard",  price: 1500, quantity: 10 },
    { id: 102, name: "mouse",     price: 600,  quantity: 25 },
    { id: 103, name: "headphone", price: 2200, quantity: 8  },
];
console.log(productsList.length);
```

Expected output

```
3
```

Now try adding an object with unrelated keys (for example email and phone number):

```
productsList.push({ email: "a@b.com", phone: "99999" });   // ERROR
```

Editor / compiler error

```
Object literal may only specify known properties, and 'email' does not exist in type 'Product'.
```

Because of the type safety, nothing except a valid **Product** can be accommodated in this array. So the interface type works together with arrays too.

6.4 Optional properties (the ? symbol)

Sometimes an object may or may not have a property - e.g. a product **description**. If you declare *description: string*, every product **must** have it. To make it optional, add a **question mark** after the property name.

```
interface Product {
    id: number;
    name: string;
    price: number;
    quantity: number;
    description?: string;       // optional property
}

const p1: Product = { id: 101, name: "keyboard", price: 1500, quantity: 10,
                      description: "Mechanical keyboard" };
const p2: Product = { id: 102, name: "mouse", price: 600, quantity: 25 };   // OK: no description

console.log(p2.description);
```

Expected output

```
undefined
```

When the key is simply not present, reading *p2.description* gives **undefined**.

Explicitly assigning undefined

```
const p3: Product = { id: 103, name: "headphone", price: 2200, quantity: 8,
                      description: undefined };    // ERROR when exactOptionalPropertyTypes is true
```

Editor / compiler error

```
Type 'undefined' is not assignable to type 'string' with 'exactOptionalPropertyTypes: true'.
Consider adding 'undefined' to the type of the target.
```

This error comes from a setting in **tsconfig.json**: *exactOptionalPropertyTypes*. If you set it to **false**, the error disappears and an optional property may explicitly be undefined (you do not have to add undefined to the type yourself). The instructor used this to show that editing tsconfig.json **changes how the compiler behaves**.

exactOptionalPropertyTypes	Omit the key	description: undefined
true (strict)	Allowed	Error
false	Allowed	Allowed

6.5 readonly properties

By default a JSON object's values can be changed later. If a property must never be changed after creation, mark it **readonly**.

```
interface Product {
    id: number;
    name: string;
    readonly price: number;      // can be read, never re-assigned
    quantity: number;
}

const product: Product = { id: 101, name: "keyboard", price: 1500, quantity: 10 };

product.name = "wireless-keyboard";   // OK  - name is a normal property
product.price = 5;                    // ERROR - price is readonly
console.log(product.name, product.price);
```

Editor / compiler error

```
Cannot assign to 'price' because it is a read-only property.
```

Expected output (after removing the erroneous line)

```
wireless-keyboard 1500
```

INSTRUCTOR'S WORDS (cleaned up)

Think about which properties you do not want to be updated or changed later - make only those **read-only**. The data is then only for reading; no one can write into it. Whether to use readonly depends entirely on the requirement.

Part 7 – Type Aliases (the type keyword)

7.1 Defining a type with the type keyword

Apart from **interface**, there is another way to define data types: **type aliasing**. The **type** keyword helps you create your own data type.

```
type Book = {
    id: number;
    name: string;
    isbn: string;       // ISBN is kept as a string
    price: number;
};

const book: Book = { id: 111, name: "My Book", isbn: "978-1-23456-789-0", price: 499 };
console.log(book.name);
```

Expected output

```
My Book
```

7.2 A type that allows only a fixed set of values

Suppose a **status** property can only be one of *approved*, *pending* or *rejected*. If you type it as **string**, any string would be accepted. To restrict it to exactly these values, create a type with the values separated by the **|** (pipe) symbol.

```
type Status = "approved" | "pending" | "rejected";

let myStatus: Status = "approved";   // OK
myStatus = "pending";                // OK
myStatus = "hello";                  // ERROR
```

Editor / compiler error

```
Type '"hello"' is not assignable to type 'Status'.
```

Only the group of values approved, pending, rejected can be placed in the variable; anything else is rejected. **This kind of restriction cannot be written with an interface - for it we use the type alias.**

> **ADDED EXPLANATION (not stated in the lecture)**
> A type that lists exact allowed values is called a **literal union type**. It is the TypeScript way to model choices (statuses, roles, directions) without using plain strings.

7.3 Interface vs type alias - which one to use?

> **INSTRUCTOR'S WORDS (cleaned up)**
> **Recommendation:** to give a shape to **objects** and **classes**, always use an **interface**. For anything else - if it is not an object and not a class - use a **type alias**.

Point	interface	type alias
Keyword	`interface`	`type`
Recommended for (lecture)	Shape of objects and classes	Everything else (e.g. fixed-value sets, unions)
Restricting to certain values (approved / pending / rejected)	Not suitable	Yes - use type
Can an object type use the other?	Yes - an interface can use a type as a property type (see example)	Yes

Using both together

```
type Status = "approved" | "pending" | "rejected";

interface IUser {
    id: number;
    userName: string;
    email: string;
    status: Status;        // uses the type alias; only approved / pending / rejected
}

const u: IUser = { id: 1, userName: "sumeet", email: "s@example.com", status: "approved" };
```

7.4 Naming conventions (recommended, not mandatory)

- Interface names start with the prefix **I**: *IUser*, *IProduct*.
- Custom type-alias names start with the prefix **T**: *TBook*, *TStatus*.

This only makes code more readable - at a glance you know whether a name is an interface or a type alias. The instructor said it is totally up to you, but that if you follow it consistently, it helps when the code is read later. (In the earlier examples the interface was written as *Product* for simplicity; by this convention it would be *IProduct*.)

```
interface IProduct { id: number; name: string; }
type TBook   = { id: number; name: string; isbn: string; price: number };
type TStatus = "approved" | "pending" | "rejected";
```

Part 8 – Union Types

Sometimes a variable can hold values of **different data types** - sometimes type 1, sometimes type 2. For this we use a **union of data types**, written with the **|** symbol.

```
let z: number = 8;
z = undefined;            // ERROR: undefined is not allowed for a number variable
```

Editor / compiler error

```
Type 'undefined' is not assignable to type 'number'.
```

If it is required, make z either a number or undefined:

```
let z: number | undefined = 8;
z = undefined;            // OK
z = 20;                   // OK
console.log(z);
```

Expected output

```
20
```

A union can have many members - the choice depends on your requirement, whether a single type or several types combined. When you use four or five data types together, give the union a name using a type alias:

```
type TUndefinedNumber = undefined | number;

let zz: TUndefinedNumber = 8;
zz = undefined;           // OK
```

Declaration	What it can hold
`let z: number`	Only numbers
`let z: number | undefined`	A number or undefined
`type Status = 'a' | 'b'`	Only the listed values

COMMON MISTAKE
With a plain *number* type, assigning *undefined* is an error. Add | *undefined* only when the value can genuinely be missing - otherwise you give up some of the safety TypeScript provides.

Part 9 – Combining Types to Reuse Properties

9.1 The repeated-properties problem

Suppose we have a person type with **name, phone and email**. Now we also need an **employee** type and a **student** type. Both employee and student have the same three properties; if we write them separately we repeat the same properties in every type. Employee additionally has a **salary** (number) and student has **marks** (number).

Instead of repeating, we create the reusable type once and **reuse an existing data type** inside another type - a kind of **inheritance** idea. Multiple types are combined to create a single type.

9.2 The solution

```
type TPerson = {
    name: string;
    phone: string;
    email: string;
};

type TEmployee = TPerson & { salary: number };
type TStudent  = TPerson & { marks: number };

const emp: TEmployee = {
    name: "Asha", phone: "9999999999", email: "asha@example.com", salary: 50000
};

const stu: TStudent = {
    name: "Ravi", phone: "8888888888", email: "ravi@example.com", marks: 88
};

console.log(emp.salary, stu.marks);
```

Expected output

```
50000 88
```

INSTRUCTOR'S WORDS (cleaned up)
Every employee object now gets these four properties: name, phone, email and salary. Similarly, every student object gets name, phone, email and marks. The person properties were written **once** and reused.

ADDED EXPLANATION (not stated in the lecture)
Operator note. The transcript text around this part is garbled (it mentions union, a pipe, and the word "and"). The behaviour the instructor described - an object that has **all** the properties of both types - is TypeScript's **intersection operator &**. A union (**|**) means *one of* the types; an intersection (**&**) means *all of* them combined.

Operator	Name	Meaning	Example
|	Union	Value can be **one of** the types	`number | undefined`
&	Intersection	Value must have **all** the properties of both types	`TPerson & { salary: number }`

COMMON MISTAKE
Missing a property now fails everywhere: an employee object without *email* is an error, because *email* is part of TPerson, which TEmployee includes.

9.3 Closing of the session

The instructor said the next (evening) session will continue with the running, compiling and building of TypeScript code. The session ended with a break before an exam.

Part 10 – Revision Material

10.1 Concise summary of all major topics

- **Express prefixes:** write /product or /admin once in *app.use(prefix, router)*; routes inside the router drop the prefix. Middleware such as verifyToken can also be applied once there.
- **Why TypeScript:** JavaScript has no type safety (no types for variables, arguments, return values, object shapes). TypeScript = JavaScript + data types.
- **Development only:** write TS during development; compile with **tsc**; deploy the resulting JS, never the TS.
- **Setup:** *npm init -y*, set type to module, install **typescript**, **tsx**, **@types/node** with **--save-dev**.
- **Dev dependencies** are excluded from production to save space (node_modules is expensive).
- **tsconfig.json:** controls compiler behaviour; create it with **npx tsc --init** (npx needed because TypeScript is installed locally).
- **Key options:** rootDir (./src), outDir (./dist), module, target, strictness flags, include (src) / exclude (node_modules).
- **Annotations:** variable: type; parameter: type;): returnType; void; number[] for arrays.
- **Interface** = shape of an object; supports optional (?) and readonly properties; missing or extra properties are errors.
- **type alias** = custom name for any type; can define fixed-value sets; recommended for non-object / non-class types.
- **Naming:** I prefix for interfaces, T prefix for types (convention).
- **Union (|)** allows one of several types; **combining types (&)** reuses properties across types.

10.2 Important definitions

Term	Definition	Purpose
TypeScript	JavaScript with added data types	Type safety during development
Type safety	Rules that stop values of the wrong type being used	Catch bugs early, at coding time
tsc	The TypeScript compiler	Converts .ts to .js
tsx	Dev-machine engine that runs TS directly	Skip manual compile-then-run (optional)
@types/node	Type definitions for Node built-in modules	Typed fs, http, os, path, etc.
devDependency	Package needed only during development	Left out of production installs
tsconfig.json	Configuration file for the TypeScript compiler	Controls what and how to compile
rootDir / outDir	Source folder / output folder	Mirror structure; dist holds the JS to deploy
interface	Describes the shape of an object or class	Consistent object structure
type alias	A name for a data type, created with the type keyword	Fixed-value sets, unions, reusable types
Union type	A type that allows one of several types (|)	Value can be of more than one type
Optional property	Property marked with ?	Key may be omitted
readonly	Property that cannot be reassigned	Protect values from change
npx	Runs commands from the project's local node_modules	Run local tools like tsc

10.3 Syntax cheat sheet

```
let x: number = 5;                              // typed variable
function sum(a: number, b: number): number {}   // typed args + return type
function show(): void {}                        // returns nothing
let arr: number[] = [1, 2, 3];                  // typed array
interface IProduct { id: number; name?: string; readonly price: number; }
type TStatus = "approved" | "pending" | "rejected";
let z: number | undefined;                      // union
type TEmployee = TPerson & { salary: number };  // combine types

npm init -y
npm i typescript tsx @types/node --save-dev
npx tsc --init
```

10.4 Key differences between related concepts

Compare	A	B
JavaScript vs TypeScript	No data types; errors appear at run time	Data types; errors flagged while coding
tsc vs tsx	Compiler producing .js files (used for builds)	Runs .ts directly on the dev machine (optional)
dependencies vs devDependencies	Needed on production	Development only
Global vs local tsc	Found by the terminal directly	Needs *npx*
rootDir vs outDir	Where TS source lives (src)	Where compiled JS goes (dist)
interface vs type	Shape of objects and classes	Everything else (fixed values, unions)
Union vs combining types	| = one of the types	& = all properties combined
Optional vs readonly	? - may be missing	readonly - cannot be changed after creation

10.5 Common mistakes to avoid

Mistake	Why it is wrong / fix
Running *tsc --init* without npx	Terminal searches globally and fails. Use *npx tsc --init*.
Deploying .ts code to production	Compile to JS and deploy JS only.
Installing typescript, tsx, @types/node as normal dependencies	Use *--save-dev* so they are left out of production.
Keeping /product in routes after adding it to app.use()	Gives /product/product/... URLs; remove the prefix from the routes.
Forgetting the colon / return type position	Annotate with *name: type*; return type goes after the parameter parentheses.
Missing a required interface property	Compiler error; add the property or mark it optional with ?
Adding unknown keys to an interface-typed object	Excess property error; define the key in the interface.
Using string when only certain values are valid	Use a literal union type: *type Status* = '*a*' | '*b*'.
Assuming tsx checks types	(Added) tsx does not type-check; rely on the editor or *tsc --noEmit*.
Confusing | with &	| = one of; & = all combined.

10.6 Interview questions and answers

Marked (L) = answer based directly on the lecture; **(A)** = additional material beyond the lecture.

Question	Answer
(L) What is TypeScript?	JavaScript with added data types; JS features still work; adds type safety at development time.
(L) Why is plain JavaScript not recommended for enterprise apps?	It lacks type safety - no types for variables, arguments, return values or object shapes.
(L) Do we deploy TypeScript code?	No. Compile to JavaScript and deploy the JS. Deploying TS would need run-time compilation.
(L) What is tsc?	The TypeScript compiler (comparable to javac in Java).
(L) What are typescript, tsx and @types/node for?	Compiler + type checker; running TS directly on the dev machine; type definitions for Node built-in modules.
(L) Why use --save-dev?	These packages are only needed in development; marking them lets DevOps leave them out of production, saving space.
(L) What does tsconfig.json do and how is it created?	Controls compiler behaviour; created with *npx tsc --init*.
(L) Why does *tsc --init* fail but *npx tsc --init* work?	TypeScript is installed locally; npx looks in the project's node_modules.
(L) What are rootDir and outDir?	Source folder (./src) and output folder for compiled JS (commonly ./dist).
(L) How do you make a property optional / readonly?	Add ? after the name for optional; prefix with readonly for non-reassignable.
(L) When to use interface vs type?	Interface for objects and classes; type alias for everything else.
(L) How do you restrict a variable to a fixed set of values?	A type alias of literal values joined by |, e.g. 'approved' | 'pending' | 'rejected'.
(A) What is the difference between | and & types?	| accepts one of the types; & requires all properties from every type.
(A) Does tsx check types?	No, it only strips them and runs; use editor / *tsc --noEmit* to type-check.
(A) What does the void return type mean?	The function returns no value.

10.7 Practice exercises (solutions not included)

Easy

- Create a new folder, run *npm init -y*, set the type to module, and install typescript, tsx and @types/node as dev dependencies.
- Generate tsconfig.json using npx, then set rootDir to ./src and outDir to ./dist.
- Declare a number, a string and a boolean variable with type annotations. Deliberately assign a wrong type to each and read the errors.
- Write a function *multiply(a, b)* with typed parameters and return type, and call it with a string to see the error.

Medium

- Create an interface for a Student with id, name, marks and an optional email. Create an array of three students.
- Mark the id of Student as readonly and prove with an error that it cannot be changed.

- Create a type *TRole* allowing only "admin", "user" or "guest" and use it inside an interface for a user.
- Set *exactOptionalPropertyTypes* to true and false and observe what happens when you assign undefined to an optional property.
- In your Express app, move the /admin prefix into app.use() and verify the old URLs still work.

Challenging

- Define TPerson (name, phone, email), then build TTeacher (adds subject) and TStudent (adds marks) by combining types, and create one object of each.
- Create a function that takes a TStatus and returns a string message for each possible value, and enable noImplicitReturns to see the compiler behaviour.
- Enable noUnusedLocals and noUncheckedIndexedAccess and write code that triggers each warning.
- Explain in your own words why dev dependencies matter for deployment and how a deployment script could skip them.

10.8 Final revision checklist

☐ I can explain why JavaScript lacks type safety and what TypeScript adds.
☐ I know TypeScript is for development; JavaScript output is what gets deployed.
☐ I know the roles of tsc, tsx and @types/node.
☐ I can install them as dev dependencies and explain why.
☐ I can create tsconfig.json with npx tsc --init and explain why npx is needed.
☐ I can explain rootDir, outDir, include, exclude and the strictness options.
☐ I can annotate variables, function parameters, return types, void and arrays.
☐ I can define an interface with optional and readonly properties.
☐ I can create type aliases, including fixed-value types.
☐ I know when to use interface and when to use type, and the I / T naming convention.
☐ I can write union types and combine types for reuse.
☐ I can move route prefixes into app.use() in Express.

10.9 Completeness check against the transcript

Topic discussed in the transcript	Covered in
Prefix for product router via app.use('/product')	Part 1.2
Same for admin router	Part 1.3
Applying verifyToken middleware once in app.use	Part 1.4
JavaScript lacks type safety (variables, args, return types, object shape, OOP)	Part 2.2
TypeScript = JS + data types; features retained	Part 2.3
TS for development only; compile and deploy JS; why not deploy TS	Part 2.4
tsc compiler; javac analogy	Part 2.5
typescript, tsx (optional), @types/node and their purposes	Part 3.1
Dev dependencies; node_modules cost; ESLint; DevOps leaving dev deps out	Part 3.2
npm init -y, type: module, --save-dev install, package.json keys	Part 3.3
tsconfig.json purpose, manual vs generated	Part 4.1 – 4.2
tsc --init error, local vs global, npx solution	Part 4.2
rootDir, outDir (dist), folder-structure example	Part 4.3

Topic discussed in the transcript	Covered in
module / target; strictness options; unused variables; switch; index access	Part 4.3
include / exclude (src, node_modules, src/**/*)	Part 4.4
Source must be .ts files inside src	Part 4.5
Typed variables; type errors	Part 5.1
Typed function parameters, return types, void; calling sum(3, 8)	Part 5.2
Typed arrays	Part 5.3
Interfaces; missing property error; array of interface; excess property error	Part 6.2 – 6.3
Optional properties; exactOptionalPropertyTypes	Part 6.4
readonly properties	Part 6.5
type alias (Book); fixed-value Status type	Part 7.1 – 7.2
Interface vs type recommendation; user example; I / T naming	Part 7.3 – 7.4
Union types (number | undefined; named union)	Part 8
Combining types (person / employee / student)	Part 9
Next session preview (run, compile, build); break before exam	Part 9.3